Hajj RAG Chatbot: System Documentation

1. General Pipeline Overview

The application is a Retrieval-Augmented Generation (RAG) pipeline built with **FastAPI** and **LangChain**. It is designed to act as a highly factual assistant for Hajj and Umrah guidance, capable of conversing in both English and Arabic.

The Request Flow:

1. **User Input:** A user sends a POST request to the /chat endpoint containing a message and their preferred language ("en" or "ar").
2. **Retrieval:** The user's message is vectorized and compared against a pre-loaded local **FAISS** (Facebook AI Similarity Search) vector database. The system retrieves the top 4 (k=4) most semantically relevant document chunks.
3. **Prompt Construction:** The retrieved context is injected into a strict system prompt. The prompt specifically instructs the model to answer *only* using the provided context, provide step-by-step instructions, and cite sources inline.
4. **Generation:** The prompt is sent to the LLM via the Groq API, which generates and returns the final response.

---

2. Models Used

2.1. The Large Language Model (LLM)

- **Model Name:** llama-3.3-70b-versatile
- **Provider:** Meta (Inference powered by **Groq**)
- **Role in Pipeline:** Acts as the reasoning and generation engine, synthesizing the retrieved context into a coherent, language-appropriate answer.

Technical Details & Why It Fits:

- **Architecture & Performance:** This is Meta's highly advanced 70-billion parameter transformer model, featuring Grouped-Query Attention (GQA) for optimized scalability. It rivals the quality of massive 405B parameter models in reasoning, instruction following, and multilingual capabilities.
- **Extreme Inference Speed:** Because it is hosted on Groq's specialized Language Processing Units (LPUs), it generates text at blazing speeds of roughly 276 to 280 tokens per second. This ensures your chatbot feels incredibly responsive.
- **Your Configurations:** * temperature=0.1: This is a crucial setting for a RAG pipeline. A near-zero temperature forces the model to be highly deterministic and factual, preventing it from hallucinating or "getting creative" with religious guidance.
    - max_tokens=1024: Ensures the responses are comprehensive but capped at a reasonable length to control API costs and response times.

2.2. The Embedding Model

- **Model Name:** sentence-transformers/paraphrase-multilingual-mpnet-base-v2
- **Provider:** Hugging Face
- **Role in Pipeline:** Converts user text and the underlying corpus into mathematical vectors (embeddings) so the FAISS database can perform similarity searches.

Technical Details & Why It Fits:

- **Multilingual Powerhouse:** This model was specifically trained on parallel data across 50+ languages. Because your app requires both Arabic and English support, this model seamlessly understands semantic similarities across both languages without needing separate embedding models.
- **Vector Space:** It maps sentences and paragraphs into a dense 768-dimensional vector space.
- **Pooling Mechanism:** It utilizes a mean-pooling technique on top of an XLM-RoBERTa architecture, which makes it exceptionally accurate for semantic search and finding related texts even if the exact keywords do not match.

---

3. Key Pipeline Components & Design Choices

- **Asynchronous Lifespan Management (@asynccontextmanager):** You have implemented a lifespan handler to load the heavy embedding model and the FAISS database into memory *only once* when the server starts. This is a best practice in FastAPI, preventing the server from reloading models on every API call and virtually eliminating latency during chat requests.
- **Bilingual System Prompts:** The pipeline dynamically switches between English and Arabic system prompts based on the request. Both prompts enforce strict "grounding" (telling the AI to say "I don't know" if the context lacks the answer) and mandate inline source citations ([source]).
- **Deserialization Warning:** Your FAISS.load_local function uses allow_dangerous_deserialization=True. This is necessary for loading local pickle files in newer versions of LangChain, but you should ensure that the vector_db folder is strictly generated by you and not downloaded from untrusted sources.
